Local Search for Combinatorial Optimization

IWRI Workshop day 2

Dr. Stefaan Haspeslagh

Vives & KU Leuven KULAK

April 8, 2026

Outline

1. Introduction and Motivation
2. Generic Local Search Template
3. Pivoting in Local Search
   - First-improvement
   - Best-improvement
   - Random-improvement
   - Comparison
   - Running example
4. Design decisions
   - Representation
   - Objective and Constraints
   - Neighbourhoods and Filtering
   - Other Design Decisions

Roadmap

1. Introduction and Motivation
2. Generic Local Search Template
3. Pivoting in Local Search
   - First-improvement
   - Best-improvement
   - Random-improvement
   - Comparison
   - Running example
4. Design decisions
   - Representation
   - Objective and Constraints
   - Neighbourhoods and Filtering
   - Other Design Decisions

What is Local Search?

- Family of *incomplete* methods that iteratively modify a candidate solution via small **moves**.
- Goal: quickly reach high-quality solutions on large, real-world instances.
- **Anytime**: keeps a best-so-far solution available at all times.

Core Idea

Start from an initial solution x and explore a neighbourhood $\mathcal{N}(x)$. If a suitable move $m \in \mathcal{N}(x)$ is found, set $x \leftarrow m(x)$. Under strict improvement, stop at a local optimum. Metaheuristics allow occasional non-improving steps to escape local minima.

Roadmap

1. Introduction and Motivation
2. Generic Local Search Template
3. Pivoting in Local Search
   - First-improvement
   - Best-improvement
   - Random-improvement
   - Comparison
   - Running example
4. Design decisions
   - Representation
   - Objective and Constraints
   - Neighbourhoods and Filtering
   - Other Design Decisions

Template

```
def local_search(x0, N, pivot, stop):
    x = x0; best = x
    while not stop(x, best):
        M = list(N(x))
        m = pivot(M, x)
        if m is None:
          return best
        x = m.apply(x)
        if f(x) < f(best):
          best = x
    return best
```

Roadmap

1. Introduction and Motivation
2. Generic Local Search Template
3. Pivoting in Local Search
   - First-improvement
   - Best-improvement
   - Random-improvement
   - Comparison
   - Running example
4. Design decisions
   - Representation
   - Objective and Constraints
   - Neighbourhoods and Filtering
   - Other Design Decisions

What is a pivot rule?

Definition

A *pivot rule* chooses which admissible move to apply next from the current neighbourhood.

- Shapes the search trajectory, convergence speed, and solution quality.
- We compare three classic strategies on the same template.

Code: first-improvement pivot

```python
import random

def pivot_first_improvement(N, x):
    """Return the first improving move in randomized scan order, else None."""
    # Randomize to avoid systematic bias/cycling
    candidates = list(N(x))
    random.shuffle(candidates)
    for m in candidates:
        d = delta(m, x).          # delta = f(apply(m,x)) - f(x)
        if d < 0:                 # strict improvement acceptance
            return m
    return None                   # local optimum under first-improvement

# Usage:
# best = local_search_template(x0, N, pivot_first_improvement, stop)
```

First-improvement: Intuition and complexity

- Scan candidates in some order and take the **first** admissible improvement.
- Per-iteration cost: $O(\mathbb{E}[\tau] \cdot T_\Delta)$ where τ is the index of the first improving move.
- Pros: cheap iterations; good for large neighbourhoods with fast delta-eval.
- Cons: myopic; may accept small improvements and miss larger ones nearby.

What is τ?

Definition (first-improvement)

In one iteration, we scan candidate moves in some order and stop at the **first** improving move. Let τ be the **index** of that first improver in the scan order:

$$\tau = \begin{cases} 1, & \text{if the first improves,} \\ 7, & \text{if the 7th is the first improver,} \\ \infty \text{ (or } |N|), & \text{if none improves.} \end{cases}$$

Interpretation: $\mathbb{E}[\tau]$ is the expected number of candidates we *examine* before we can stop, i.e., before we hit an improving move.

Why $O(\mathbb{E}[\tau] \cdot T_\Delta)$ for first-improvement?

One iteration does: generate candidates $\rightarrow$ <u>scan until first improver</u> $\rightarrow$ apply move ($\rightarrow$ update caches)

Let T_Δ be the average time to compute one $\Delta(m, x)$, and $N = |\mathcal{N}(x)|$. The number of Δ-evaluations in an iteration is $\min(\tau, N)$, hence

$$\mathbb{E}[\text{iteration time}] \approx \underbrace{\mathbb{E}[\min(\tau, N)]}_{\text{expected \# evals}} \cdot \underbrace{T_\Delta}_{\text{time per eval}} + \underbrace{T_{\text{apply}} + T_{\text{update}} + T_{\text{gen}}}_{\text{smaller overhead}}.$$

Code: best-improvement pivot

```python
def pivot_best_improvement(N, x):
    """Return the steepest improving move (lowest delta), else None."""
    best_m = None
    best_d = 0.0  # improvements are negative; keep most negative
    for m in N(x):
        d = delta(m, x)
        if d < best_d:
            best_m, best_d = m, d
    return best_m  # None if no improving move exists

# Usage:
# best = local_search_template(x0, N, pivot_best_improvement, stop)
```

Best-improvement: Intuition and complexity

- Evaluate **all** candidates and take the **steepest descent** move.
- Per-iteration cost: $O(|\mathcal{N}(x)| \cdot T_{\Delta})$.
- Pros: strong single-step progress; fewer iterations to a local optimum.
- Cons: expensive scans; pair with filtered neighbourhoods/candidate lists.

Code: random-improvement pivot

```python
import random

def pivot_random_improvement(N, x, budget=128):
    """Sample up to 'budget' random candidates; return the first improving move or None.
    Increase 'budget' if improvements are sparse; decrease for quicker iterations.
    """
    # If N(x) is indexable, do O(1) random access; otherwise sample from a list copy
    candidates = list(N(x))
    n = len(candidates)
    if n == 0:
        return None
    for _ in range(min(budget, n)):
        m = candidates[random.randrange(n)]
        d = delta(m, x)
        if d < 0:
            return m
    return None  # none found within budget

# Usage:
# best = local_search_template(x0, N, lambda N,x: pivot_random_improvement(N,x,64), stop)
```

Random-improvement: Intuition and complexity

- Sample candidates uniformly until an improvement is found.
- Per-iteration cost: $O(\mathbb{E}[K] \cdot T_{\Delta})$ where K is the number of samples to first improvement.
- Pros: inexpensive in huge neighbourhoods; adds diversification; easy to parallelize sampling.
- Cons: variance in progress; needs a budget/cap to avoid long streaks without success.

When to use which?

First-improvement

- Very large neighbourhoods (candidate sets)
- $O(1)$ Δ-eval (e.g., 2-opt in TSP)
- Randomized scan order

Best-improvement

- Small, high-quality candidate sets
- Prefer strong steps; scans are costly
- Good with acceptance overlays: see SA!

Random-improvement

- Massive or streaming candidate sets
- Desire for diversification
- Tune sampling cap B adaptively

Rule of thumb

- Start with first-improvement + filtering
- If progress stalls, try best-improvement
- Add random-improvement for diversification

Running example: setup (5-city TSP, 2-opt)

Distance matrix (symmetric, cities 1–5):

$$d = \begin{pmatrix} 0 & 5 & 4 & 2 & 9 \\ 5 & 0 & 8 & 5 & 14 \\ 4 & 8 & 0 & 6 & 5 \\ 2 & 5 & 6 & 0 & 7 \\ 9 & 14 & 5 & 7 & 0 \end{pmatrix}$$

Start: $\pi = (1, 2, 3, 4, 5)$, $\quad L = d_{12} + d_{23} + d_{34} + d_{45} + d_{51} = 5 + 8 + 6 + 7 + 9 = \mathbf{35}$

All 2-opt neighbours (moves (i, j) remove edges (π_i, π_{i+1}) and (π_j, π_{j+1}), reverse segment):

Move	New tour	$\Delta L = d_{\pi_i,\pi_j} + d_{\pi_{i+1},\pi_{j+1}} - d_{\pi_i,\pi_{i+1}} - d_{\pi_j,\pi_{j+1}}$
$(1, 3)$	$(1, \mathbf{3}, \mathbf{2}, 4, 5)$	$4+5-5-6 = -\mathbf{2}$
$(1, 4)$	$(1, \mathbf{4}, \mathbf{3}, \mathbf{2}, 5)$	$2+14-5-7 = +4$
$(2, 4)$	$(1, 2, \mathbf{4}, \mathbf{3}, 5)$	$5+5-8-7 = -\mathbf{5}$
$(2, 5)$	$(1, 2, \mathbf{5}, \mathbf{4}, \mathbf{3})$	$14+4-8-9 = +1$
$(3, 5)$	$(1, 2, 3, \mathbf{5}, \mathbf{4})$	$5+9-6-9 = -\mathbf{1}$

Running example: iteration 1

First-improvement

Scan in order; **stop** at first $\Delta < 0$:

Move	ΔL	
$(1,3)$	-2	$\leftarrow$ **accept & stop**
$(1,4)$	not examined	
$(2,4)$	not examined	
$(2,5)$	not examined	
$(3,5)$	not examined	

Apply $(1,3)$: $\pi \leftarrow (1,3,2,4,5)$

$L = 4+8+5+7+9 = \mathbf{33}$ (**1 Δ-eval**)

Best-improvement

Scan *all* moves; keep steepest:

Move	ΔL	
$(1,3)$	-2	candidate
$(1,4)$	$+4$	skip
$(2,4)$	-5	**new best**
$(2,5)$	$+1$	skip
$(3,5)$	-1	skip

Apply $(2,4)$: $\pi \leftarrow (1,2,4,3,5)$

$L = 5+5+6+5+9 = \mathbf{30}$ (**5 Δ-evals**)

FI accepted a weaker move ($\Delta = -2$ vs. $\Delta = -5$) but used only **1** evaluation instead of 5.

Running example: iteration 2 and convergence

FI from $(1, 3, 2, 4, 5)$, $L = 33$

Move	ΔL	
$(1, 3)$	$+2$	skip
$(1, 4)$	-4	$\leftarrow$ **accept & stop**
$(2, 4)$	not examined	
$(2, 5)$	not examined	
$(3, 5)$	not examined	

Apply $(1, 4)$: $\pi = (1, 4, 2, 3, 5)$
$L = \mathbf{29}$ (2 evals; *still improving*)

BI from $(1, 2, 4, 3, 5)$, $L = 30$

Move	ΔL	
$(1, 3)$	-1	candidate
$(1, 4)$	$+8$	skip
$(2, 4)$	$+5$	skip
$(2, 5)$	$+9$	skip
$(3, 5)$	-4	**steepest**

Apply $(3, 5)$: $\pi = (1, 2, 4, 5, 3)$
$L = \mathbf{26}$ (5 evals; *local optimum!*)

After 2 iterations: BI at local optimum $L = 26$ (10 Δ-evals total); FI at $L = 29$ and still iterating (3 evals so far).

Key take-away: BI uses more evaluations per iteration but reaches a better local optimum in fewer iterations.

Roadmap

1. Introduction and Motivation
2. Generic Local Search Template
3. Pivoting in Local Search
   - First-improvement
   - Best-improvement
   - Random-improvement
   - Comparison
   - Running example
4. Design decisions
   - Representation
   - Objective and Constraints
   - Neighbourhoods and Filtering
   - Other Design Decisions

Representation: general goals and patterns

Goals

Enable cheap local edits and fast Δ-evaluation by storing the right local context.

- Constant-time access to predecessors, successors, neighbours, ...
- Splice, reverse, or insert in $O(1)$ or proportional to edited segment length.
- Maintain cached aggregates so that $\Delta(m, x)$ is $O(1)$ when possible.

Common pattern for TSP

Permutation with inverse index

Representation: TSP

Encoding: permutation $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ of cities; tour visits them in order and returns to π_1.
Caches:

- **Position array** $\mathrm{pos}[c]$: index of city c in π for $O(1)$ lookup.
- **Predecessor/successor arrays** $\mathrm{prev}[c]$, $\mathrm{next}[c]$: for $O(1)$ neighbour access.
- **Total tour cost** $L = \sum_{i=1}^{n} d(\pi_i, \pi_{i+1 \bmod n})$.

Example (5 cities, cost matrix d):

$$\pi = (1, 3, 5, 2, 4) \;\Rightarrow\; L = d_{1,3} + d_{3,5} + d_{5,2} + d_{2,4} + d_{4,1}.$$

Δ-update for 2-opt: reversing segment $[\pi_i, \ldots, \pi_j]$ costs $O(1)$ to evaluate (subtract two old edges, add two new ones); $O(j - i)$ to apply the reversal in a plain array, $O(\log n)$ with a splay-tree structure.

Objective and constraints: general choices

Two philosophies

Feasible-only moves (preserve constraints) vs **penalised** objective $F(x) = f(x) + \sum_k \lambda_k \operatorname{viol}_k(x)$.

Adaptive penalties

Increase λ_k while constraint k is violated; decrease otherwise. $\lambda_k \leftarrow \lambda_k \times \alpha_+$ if violated, else $\times \alpha_-$ with $\alpha_+ > 1 > \alpha_- > 0$.

Objective and constraints: TSP

Objective: minimize tour length $L(\pi) = \sum_{i=1}^{n} d(\pi_i, \pi_{i+1 \bmod n})$.

Constraints: visit every city exactly once — satisfied automatically by the permutation encoding (no penalty needed if moves preserve the permutation).

Numeric example (4 cities, symmetric distances):

$$d = \begin{pmatrix} 0 & 2 & 9 & 10 \\ 2 & 0 & 6 & 4 \\ 9 & 6 & 0 & 8 \\ 10 & 4 & 8 & 0 \end{pmatrix}$$

Tour $\pi = (1, 2, 3, 4)$: $L = d_{12} + d_{23} + d_{34} + d_{41} = 2 + 6 + 8 + 10 = 26$.
Tour $\pi' = (1, 2, 4, 3)$: $L = 2 + 4 + 8 + 9 = 23$ — better by $\Delta L = -3$.

Penalty form (optional, e.g., if edge-constraints are added): $F(\pi) = L(\pi) + \lambda \cdot \mathrm{viol}(\pi)$.

Edge-constraints and penalties: TSP examples

Two common types of edge-constraint:

- **Forbidden edge** (i, j): that pair may *not* be adjacent in the tour (road closure, incompatibility).
- **Required edge** (i, j): that pair *must* be adjacent (fixed delivery link, pre-committed leg).

Penalised objective: $F(\pi) = L(\pi) + \lambda \cdot \mathrm{viol}(\pi)$, $\mathrm{viol}(\pi) =$ number of violated edge-constraints.

Numeric example (same 4-city instance; **forbidden edge** $(1, 2)$):

Tour π	Edges used	$L(\pi)$	viol	$F = L + \lambda \cdot \mathrm{viol}$
$(1, 2, 3, 4)$	(1,2), (2,3), (3,4), (4,1)	26	1	$26 + \lambda$
$(1, 2, 4, 3)$	(1,2), (2,4), (4,3), (3,1)	23	1	$23 + \lambda$
$(1, 3, 2, 4)$	$(1, 3)$, $(3, 2)$, $(2, 4)$, $(4, 1)$	29	0	**29**
$(1, 4, 2, 3)$	$(1, 4)$, $(4, 2)$, $(2, 3)$, $(3, 1)$	29	0	**29**

- $\lambda = 10$: feasible tours win ($F = 29$) despite higher raw cost — **correct behaviour**.
- $\lambda = 2$: $(1, 2, 4, 3)$ gives $F = 25 < 29$ — **under-penalisation** lets infeasible solutions dominate.
- Rule of thumb: set λ large enough that feasibility is always preferred over raw cost savings.

Neighbourhoods and filtering: general principles

Balance power vs cost

Large neighbourhoods improve quality but are expensive to scan; use filtering to keep $N = |\mathcal{N}(x)|$ small.

- Candidate lists.
- Granular rules.

Neighbourhoods: TSP

Key moves (all preserve the permutation constraint):

- **2-opt:** remove two edges (π_i, π_{i+1}) and (π_j, π_{j+1}), reconnect by reversing the segment $[\pi_{i+1}, \ldots, \pi_j]$.
$$\Delta L = d_{\pi_i,\pi_j} + d_{\pi_{i+1},\pi_{j+1}} - d_{\pi_i,\pi_{i+1}} - d_{\pi_j,\pi_{j+1}}$$
  Size: $O(n^2)$ moves; Δ-eval in $O(1)$.
- **3-opt:** remove three edges and reconnect; up to 8 reconnection patterns. Stronger but $O(n^3)$ neighbourhood.
- **Or-opt:** relocate a chain of 1, 2, or 3 consecutive cities to a different position. Cheaper than 3-opt; $O(n^2)$ moves.
- ...

Rule of thumb for TSP: start with 2-opt + candidate lists; add Or-opt for diversity

TSP: candidate lists

Idea. For each city c, pre-compute its k nearest neighbours ($k \approx 5$–20). Only consider 2-opt moves that use an edge to a near neighbour.

Candidate list $\mathcal{CL}(c)$: the k cities closest to c by $d_{c,\cdot}$, sorted ascending. Built once; takes $O(kn)$ space.

Filtered 2-opt scan:

- For each city π_i, only try partners $\pi_j \in \mathcal{CL}(\pi_i)$.
- **Early termination:** if $d_{\pi_i,\pi_j} \geq d_{\pi_i,\pi_{i+1}}$ (the new edge is already longer than the removed one), all remaining candidates in $\mathcal{CL}(\pi_i)$ are worse — stop.

Complexity impact (2-opt):

$$O(n^2) \text{ naive} \quad \longrightarrow \quad O(kn) \text{ with candidate lists.}$$

Rule of thumb: $k = 5$ recovers most of the quality of full 2-opt in a fraction of the time on Euclidean instances.

TSP: granular rules (cheap pre-filters for 2-opt)

Purpose. Discard weak move candidates before computing full ΔL; keep scans tiny and focused.

Distance threshold (granularity bound):

- A 2-opt move swapping edges $(i, i{+}1)$ and $(j, j{+}1)$ can only improve the tour if

$$d_{\pi_i,\pi_j} < d_{\pi_i,\pi_{i+1}}.$$

- *Granular rule:* skip pair (i, j) if $d_{\pi_i,\pi_j} \geq (1+\sigma) \cdot d_{\pi_i,\pi_{i+1}}$ for a small slack $\sigma \geq 0$.

Positive-gain cut:

- As soon as the gain $g_1 = d_{\pi_i,\pi_{i+1}} - d_{\pi_i,\pi_j} \leq 0$, the candidate list is exhausted (sorted by distance); break inner loop.

Don't-look bits:

- Mark city π_i as "don't look" after a full scan finds no improvement involving it.
- Clear the bit whenever an adjacent edge changes. Reduces amortized scan cost substantially.

Initialisation

What A procedure that builds the *starting solution* x_0 for local search.

Why it matters

- Better $x_0 \Rightarrow$ fewer iterations to reach a strong local optimum.
- The neighbourhood structure can have many basins; x_0 steers which basin you hit.

Design choices

- *Constructive*: fast greedy/insertion rules; typically feasible-by-design.
- *Randomized*: perturb constructive steps to diversify multi-starts.
- *Guided*: round LP relaxations

Interfaces with LS

- Feasible-only LS needs feasible x_0; penalty-based LS can accept slightly infeasible x_0.
- Record random seeds and heuristic parameters for reproducibility.

Warm vs. Cold start

Warm start = LS begins from a solution built by a fast heuristic.

- TSP: nearest-neighbour / greedy insertion.

Cold start = LS begins from an uninformed baseline (random/trivial), not tuned to the structure.

- TSP: random permutation of cities (or random 2-edge-connected tour).

Trade-offs

- Warm: faster convergence, better early quality, less variance (intensification).
- Cold: greater basin diversity, useful for benchmarking and diversification, but slower.

Initialisation: TSP

Nearest-neighbour (NN) heuristic (warm start)

- Start from a random city; repeatedly visit the nearest unvisited city; close the tour.
- Fast: $O(n^2)$ naively, $O(n \log n)$ with a k-d tree.
- Produces tours roughly 20–25% above optimal on Euclidean instances; a useful warm start.

Greedy edge insertion

- Sort all edges by length; add each edge if it does not create a vertex of degree > 2 or a subtour (until a Hamiltonian cycle is formed).
- Typically better than NN; $O(n^2 \log n)$.

Tiny example (5 cities; start at city 1):

$$\text{NN tour: } 1 \to 2 \to 4 \to 3 \to 5 \to 1.$$

Cold start: random permutation, then apply 2-opt immediately to repair obvious crossings.
Multi-start: repeat NN from every city; keep shortest as π_0.

Stopping, tuning, reproducibility

- Stopping: time limit, iteration budget, no-improvement window, ...
- Tuning: all the parameters. A search problem on its own!
- Reproducibility: fix seeds; log instance IDs, parameters, solver version; report distributions (median).

Acceptance criteria - see further :-)

- Here: strict improvement (hill climbing): accept iff $\Delta < 0$.
- Tabu search: forbid recent move attributes for tenure τ; aspiration overrides; kind of memory.
- Simulated annealing: accept with probability $\exp(-\Delta/T)$; geometric cooling.